

Textures add surface details to 3D objects, enhancing realism, depth, and visual appeal by simulating intricate patterns, colours, and material properties, making 3D objects more immersive and lifelike in web applications.



Creating a visually compelling 3D scene goes beyond just placing objects—it requires textures, materials, lighting, and a well-defined background to bring everything together. Textures add surface details, materials control how objects interact with light, and proper lighting enhances depth and realism.

In this lesson, we'll see how textures add details, making surfaces rough like bricks or smooth like glass. We'll explore matte, glossy, transparent materials and more, and see how lighting adds depth and realism.

By the end of this lesson, we will understand how these elements work together to create immersive 3D scenes in Three.js.



Activity

Create a cube and apply textures, materials, lights, and shadows for a realistic look.

Part 1 - Setup

Let us use starter files to get started quickly without writing everything from scratch. Download them by scanning the QR code.

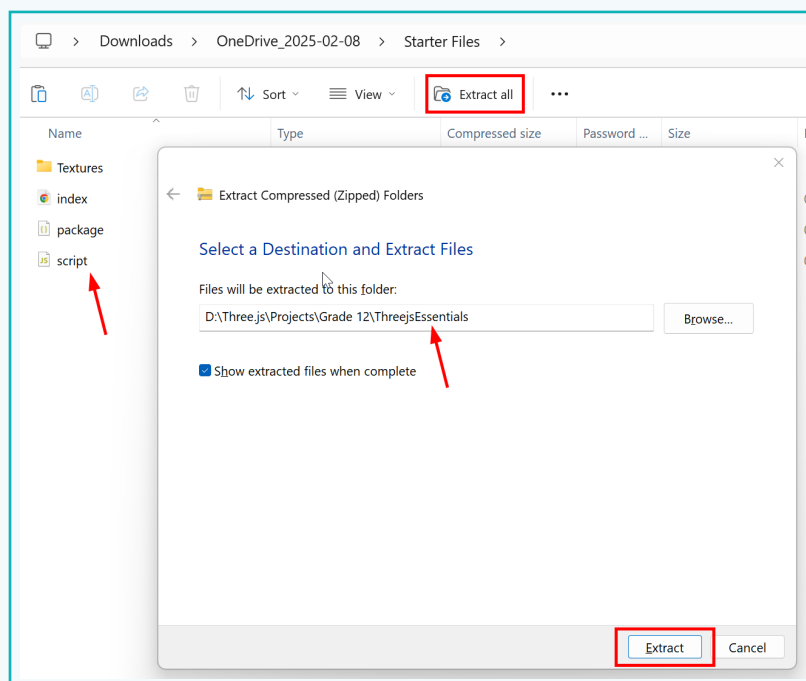


Dive into the video for a walkthrough of the starter file's code - it is both a guide and a quick refresher on everything we've explored so far!

Scan the given QR code to access it.

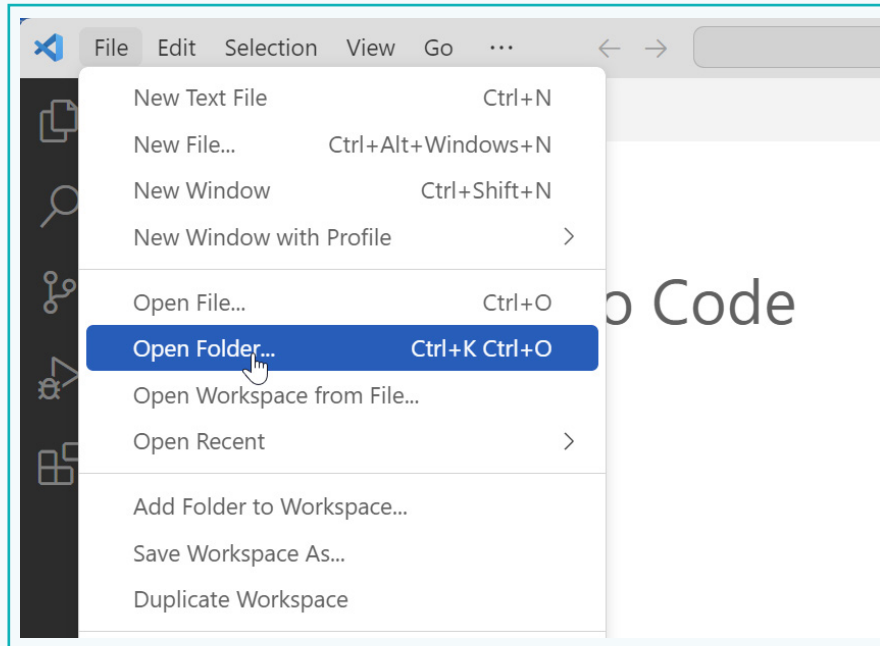


- 1 Create a new folder and extract the starter files into this folder.

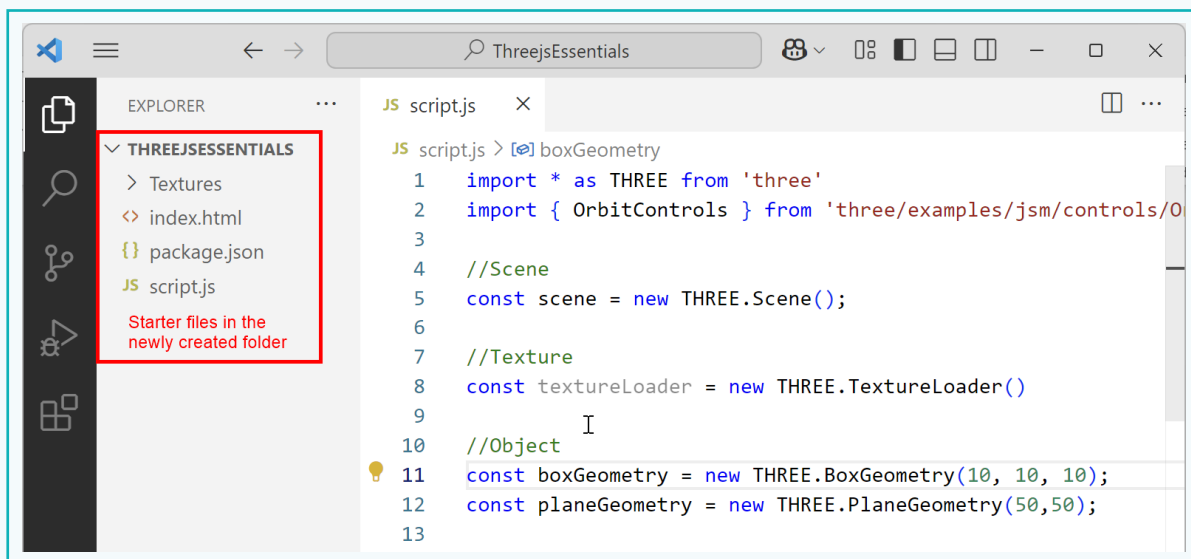


Activity

- 2 Open **VSCode** and click on 'File -> Open Folder'. Navigate to the folder where the starter files have been extracted and click on 'Open'.



This is how the files should appear.



Activity

Let us quickly review the contents of the starter folder.

- **Textures** – Contains the texture files used for materials.
- **index.html** – The main HTML file that sets up the webpage and includes the necessary scripts for running the Three.js scene.
- **script.js** – The JavaScript file where the Three.js scene is created, including objects, materials, lighting, and rendering logic.
- **package.json** – Contains information about the dependencies and packages required for the project. This file can be created by running the command `npm init -y`.

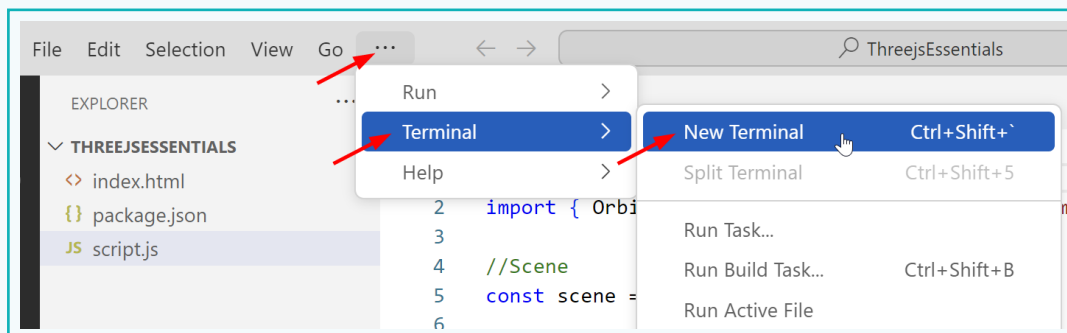
Go through the code in 'script.js' and relate each part of the code to the concepts explained in the above video.

Note:

This is the basic code setup used in most projects. Each one needs a scene, mesh (geometry + material), lights, a camera, and a renderer. These parts stay the same in every project—only the object or mesh usually changes.

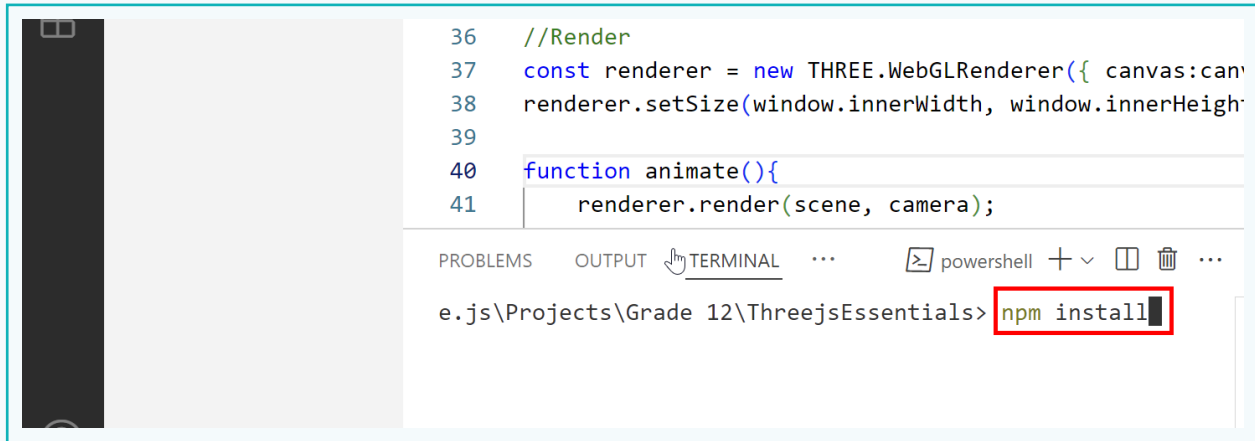
A folder that is not included with the starter files, but is required, is the 'node_modules' folder. This folder contains all the installed dependencies needed for the project. To create this folder, we must run the `npm install` command.

- 3 Open the terminal.



Activity

4 Run *npm install*.



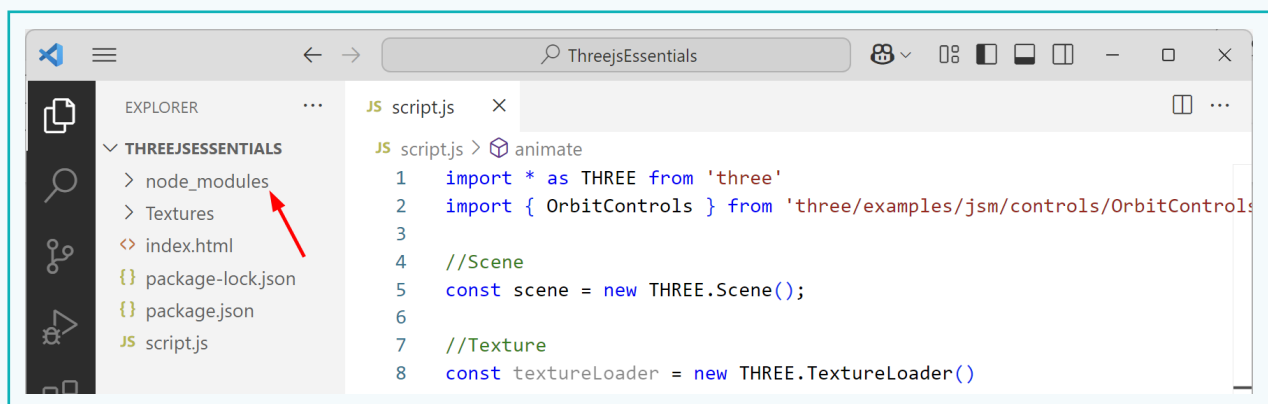
```
36 //Render
37 const renderer = new THREE.WebGLRenderer({ canvas: canvas })
38 renderer.setSize(window.innerWidth, window.innerHeight)
39
40 function animate(){
41     renderer.render(scene, camera);
42 }
```

PROBLEMS OUTPUT **TERMINAL** ... powershell + - [] [] ...

e.js\Projects\Grade 12\ThreejsEssentials> **npm install**

Executing *npm install* performs the following actions:

- **Creates the node_modules folder:** If it doesn't already exist, npm will create this directory to store all the installed packages.
- **Installs dependencies:** It reads the 'package.json' file and installs all listed dependencies, including the 'Three.js', 'Vite', and any other required packages for the project.



EXPLORER

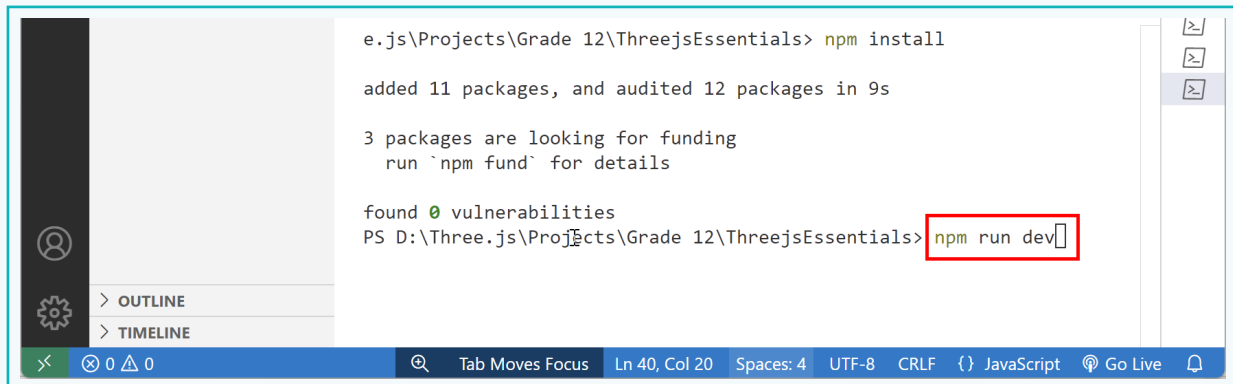
- THREEJSESENTIALS
 - node_modules
 - Textures
 - index.html
 - package-lock.json
 - package.json
 - script.js

JS script.js

```
1 import * as THREE from 'three'
2 import { OrbitControls } from 'three/examples/jsm/controls/OrbitControls'
3
4 //Scene
5 const scene = new THREE.Scene();
6
7 //Texture
8 const textureLoader = new THREE.TextureLoader()
```

Activity

- 5 Test the code by running `npm run dev`.

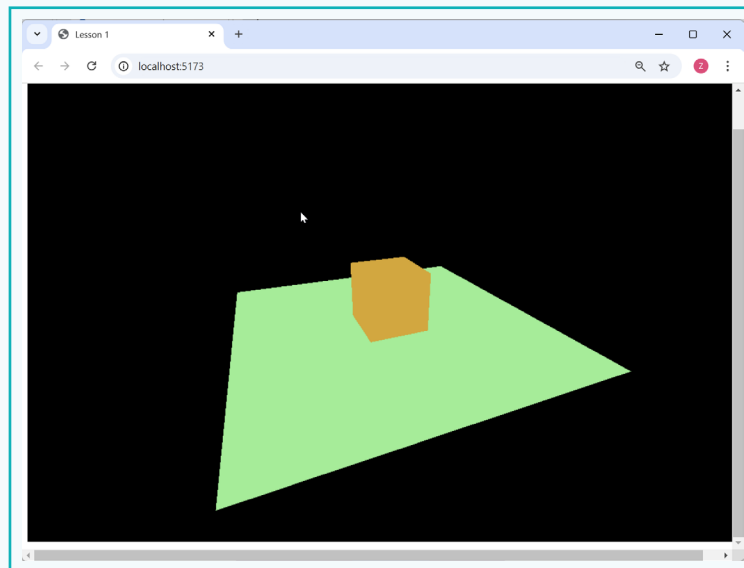


```
e.js\Projects\Grade 12\ThreejsEssentials> npm install
added 11 packages, and audited 12 packages in 9s

3 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS D:\Three.js\Projects\Grade 12\ThreejsEssentials> npm run dev
```

Output



As seen, materials have been added to the 3D objects in the scene. However, materials alone are not enough to create a realistic effect. To add depth and realism, additional elements like textures, lights, and shadows are needed.

Now, let us go beyond the basics and explore these Three.js concepts in more depth.

Activity

Part 2 - Apply Textures

In this section, we will learn what a texture is, how it adds realism and depth to objects, and finally, how to apply textures to materials.

Texture

A Texture is an image that covers the surface of a mesh. It is very useful for adding colour, patterns, and other details to a 3D model.

Different Types of Textures



Activity

How is a texture different from a material?

In 3D graphics, materials and textures are both used to define how an object looks, but they serve different purposes:

Materials

A **material** is a collection of properties that define how light interacts with an object.

It controls aspects like **colour, shininess, transparency, reflectivity, and roughness.**

Textures

A **texture** is an image (or a set of images) applied to a material to add **detail and variation.**

Textures add complexity to the surface by simulating **real-world details** like wood grain, fabric patterns, or scratches.

How to load textures?

We can load textures using the *TextureLoader* class. We will instantiate an object of the *TextureLoader* class and then use its *.load(..)* method to apply a texture to a material.

Note:

A folder named 'Textures' is present within the starter files. There are two image files in it. We will use the 'brickTexture.jpg' file for our texture.



Activity

- 1 Create an instance of the *TextureLoader* class.
- 2 Load the texture using the *.load* method of the *TextureLoader* class.

```
JS script.js X
JS script.js > ...
1 import * as THREE from 'three'
2 import { OrbitControls } from 'three/examples/jsm/controls/OrbitControls.'
3
4 //Scene
5 const scene = new THREE.Scene();
6
7 //Texture
8 const textureLoader = new THREE.TextureLoader();
9 const bricktexture = textureLoader.load('./Textures/brickTexture.jpg')
10
11 //Object
12 const boxGeometry = new THREE.BoxGeometry(10, 10, 10);
13 const planeGeometry = new THREE.PlaneGeometry(50,50);
14
15 //Material
```

A single *TextureLoader* instance can load multiple textures as needed.

To see the texture on the cube, we must replace the material's *color* property with the *map* property and use the texture as its value.

- 3 Replace the *color* property with the *map* property where the material instance is being created. Replace the *Goldenrod* property with the *bricktexture* property.

```
//Object
const boxGeometry = new THREE.BoxGeometry(10, 10, 10);
const planeGeometry = new THREE.PlaneGeometry(50,50);

//Material
const boxMaterial = new THREE.MeshBasicMaterial({color:'Goldenrod'})
const planeMaterial = new THREE.MeshBasicMaterial({color:'LightGreen'})
```

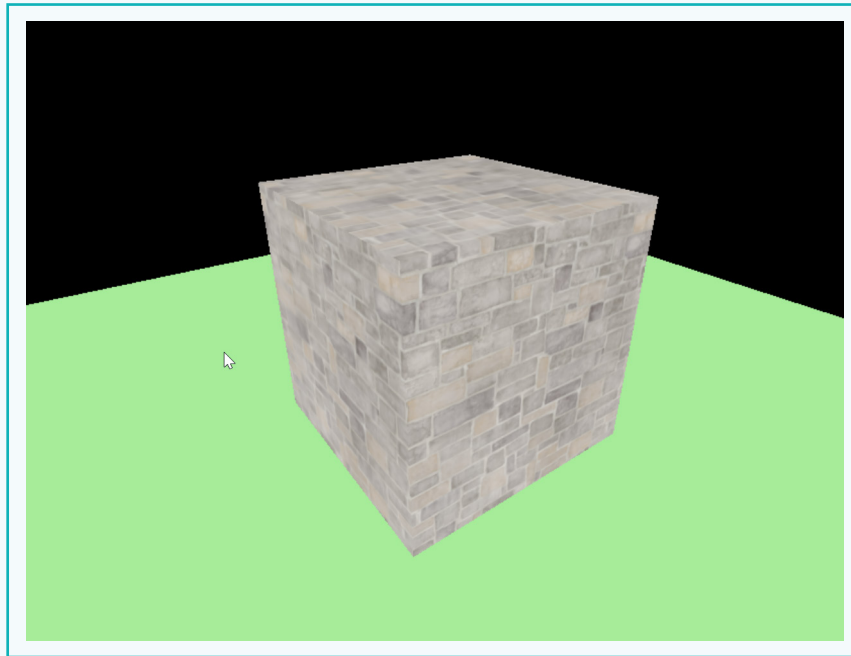
replace with *map* replace with *bricktexture*

- This is how the line of code should look.

```
//Material
const boxMaterial = new THREE.MeshBasicMaterial({map:bricktexture})
const planeMaterial = new THREE.MeshBasicMaterial({color:'LightGreen'})
```

Activity

- Go back to the browser and check the output.



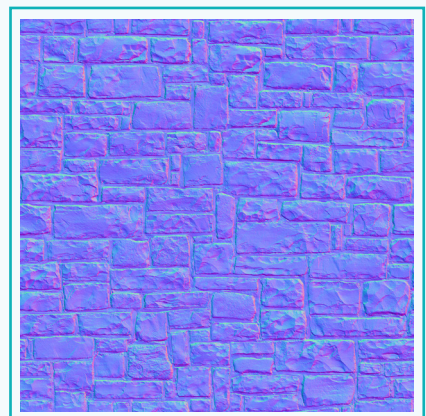
The texture is applied to the cube. However, the bricks still lack realism and appear flat. Light should reflect differently across the surface to create a more natural look. A **normal** texture achieves this variation in light reflection, adding depth and detail without altering the actual geometry.

Normal Texture

In Three.js, a *normal texture* (*normal map*) creates the illusion of surface details like bumps, dents, and grooves without modifying the actual 3D model's geometry.

It is an image where colours represent directions (X, Y, and Z) of **surface normals**, influencing how light interacts with the surface. In 3D design, 'normals' are invisible lines that point outward from a surface, showing which way it is facing. Each pixel in a normal map contains an RGB colour, where:

- Red (R) controls the X direction (left or right tilt).
- Green (G) controls the Y direction (up or down tilt).
- Blue (B) represents the Z direction (depth or outward-facing areas).



Activity

Since most surfaces in a normal map face outward along the Z-axis, the blue (B) channel is usually the most intense, giving normal maps their characteristic bluish-purple appearance.

When light hits a surface, a normal map alters the reflection, adding realistic shadows and highlights without increasing polygons.

To see this effect, we need materials that interact with light. **MeshBasicMaterial** ignores lighting, so normal maps won't work. Instead, let us use a material that supports lighting for realistic results.

Part 3 - Materials and Lights

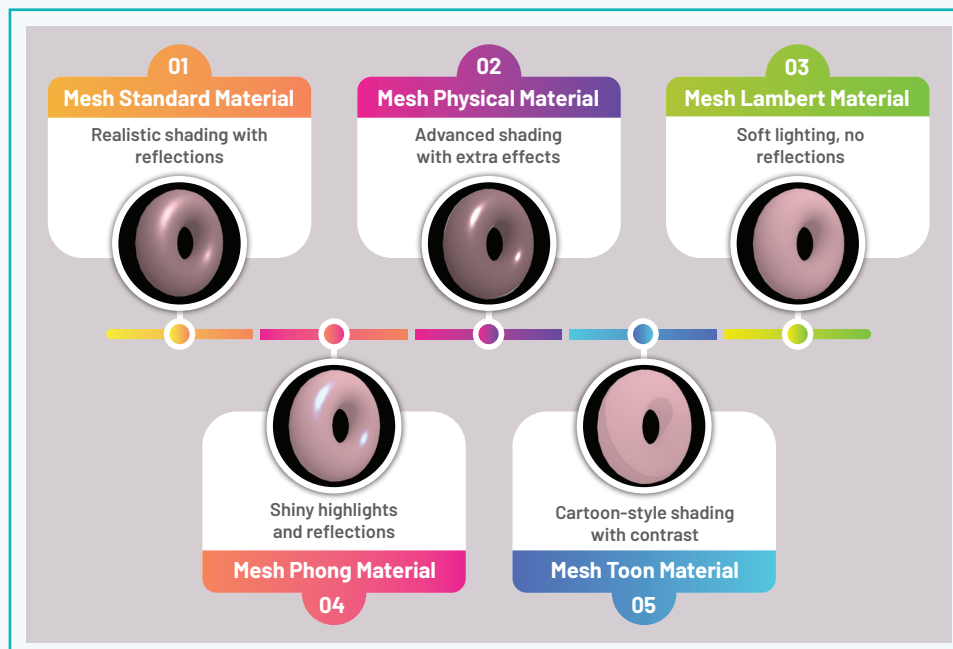
This section explores the different materials available in Three.js. Some materials rely on a light source to be visible, while others do not.

Materials that do not require lighting appear the same regardless of the scene's light setup. These include **MeshBasicMaterial**, **MeshDepthMaterial**, **MeshNormalMaterial** and **ShadowMaterial**.

We have already used **MeshBasicMaterial**. The other materials are less commonly used, so we will not cover them in detail.

Materials that require light rely on the scene lighting to appear correctly because they simulate real-world light interactions. Each of these materials uniquely interacts with light, affecting how the object appears.

Such materials are shown below:



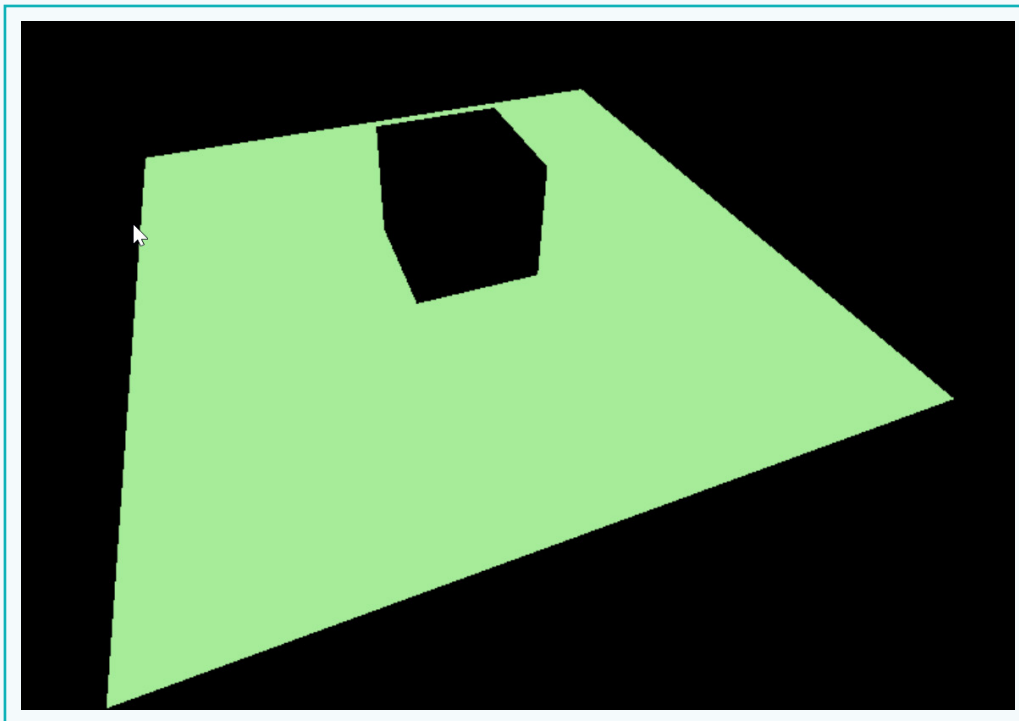
Activity

Let us apply a material that reacts to light, like *MeshStandardMaterial*, to the cube in the scene.

- 1 Change the material from *MeshBasicMaterial* to *MeshStandardMaterial*.

```
//Material  
const boxMaterial = new THREE.MeshStandardMaterial({map:bricktexture})
```

- Go back to the browser and check the output.



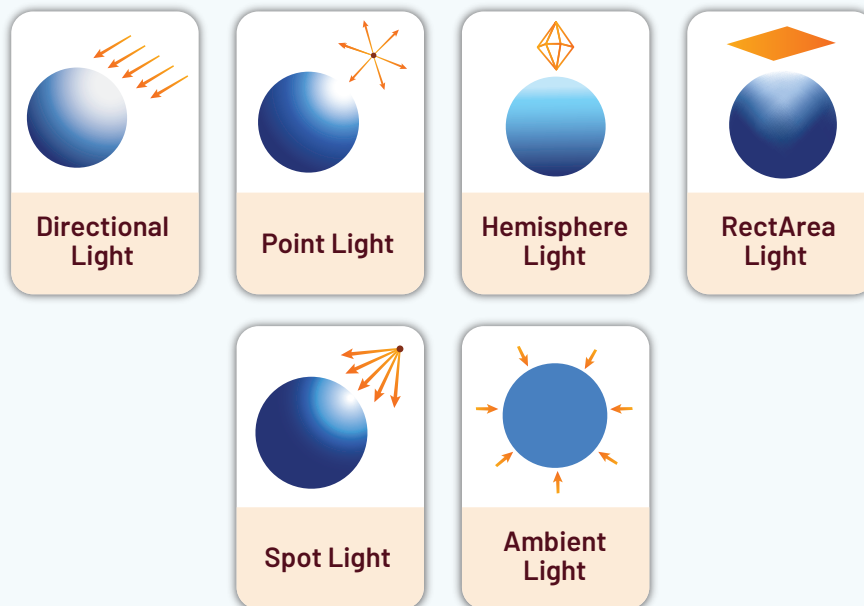
Can you figure out why the cube is appearing black?

MeshStandardMaterial requires light to be seen. It uses the lights present in the scene to light up the object on which it is applied. To understand the effectiveness of such materials, we need to have lights in the scene.

So, let us create lights.

Activity

Three.js offers different types of lights, as shown below -



Creating lights is similar to creating objects, materials, textures, etc., just like creating an instance of the respective class and then adding it to the scene.

- 2 Create an instance of the *AmbientLight* class and add it to the scene.

```
const ambientLight = new THREE.AmbientLight(0xffffff, 0.3)
scene.add(ambientLight)

//Camera
const camera = new THREE.PerspectiveCamera(75, window.innerWidth/window.innerHeight);
camera.position.z = 25;
scene.add(camera);
```

The properties passed to *THREE.AmbientLight(color, intensity)* are –

- **Color** – defines the light's colour.
In the image above,
 - **0x** - Indicates that the number is in hexadecimal (base 16) format.
 - **ffffff** - Represents white colour in RGB format.
- **Intensity** – controls the intensity of the ambient light. The default is 1.

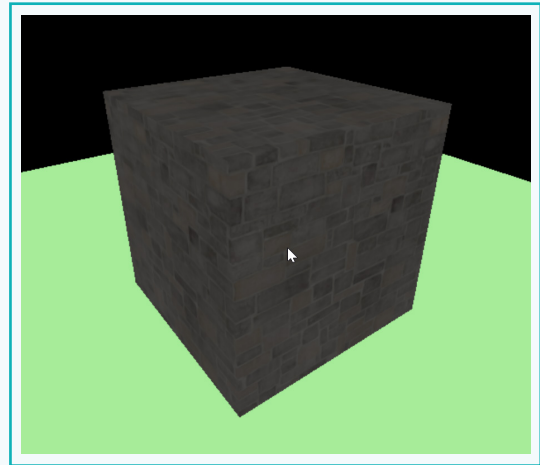
Activity

Note:

Colour names can also be specified instead of the hex codes. Refer online resources for the same.

- Go back to the browser and check the output.

The texture is now visible. The ambient light illuminates the objects in the scene equally, without casting shadows. It helps create a base level of brightness. You can experiment with the intensity values.



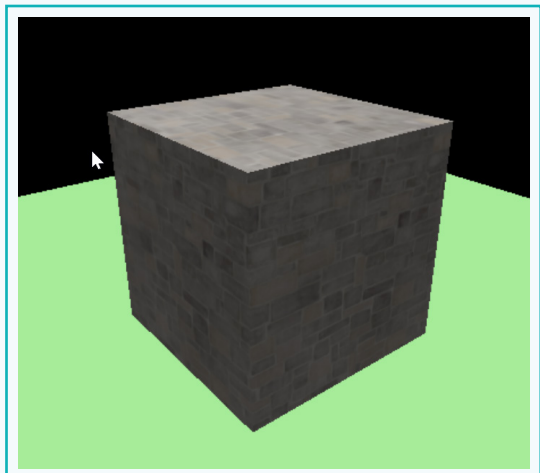
- 3 Add another light – the directional light to the scene.

```
const ambientLight = new THREE.AmbientLight(0xffffff, 0.3)
scene.add(ambientLight)

const directionalLight = new THREE.DirectionalLight(0xffffff, 2)
scene.add(directionalLight)
```

- Go back to the browser and check the output.

The directional light is coming from the top; hence the top face is illuminated.



Activity

- 4 Set the position of the directional light using the *position.set(x,y,z)* method. Experiment with different x, y & z values to understand the positioning of the light.

```
const directionalLight = new THREE.DirectionalLight(0xffffff, 2)
directionalLight.position.set(2,8,5) ←
scene.add(directionalLight)
```

Now that we have the appropriate material and lights set up to use a normal texture, we can now apply it to our 3D object.

In the 'Textures' folder, along with the *brickTexture.jpg* file, there is another image named *brickTextureNormal.png*, which represents the normal map for the brick texture.

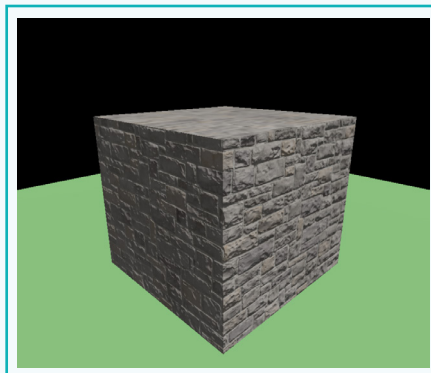
- 5 Load the normal texture using the *.load* method of the *TextureLoader* class.

```
//Texture
const textureLoader = new THREE.TextureLoader();
const bricktexture = textureLoader.load('./Textures/brickTexture.jpg');
const bricknormaltexture = textureLoader.load('./Textures/brickTextureNormal.png');
```

- 6 Assign the loaded normal texture to the *normalMap* property.

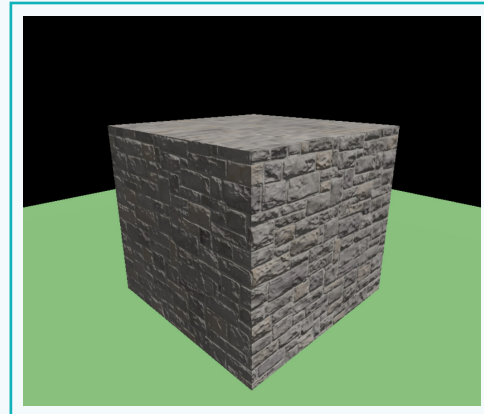
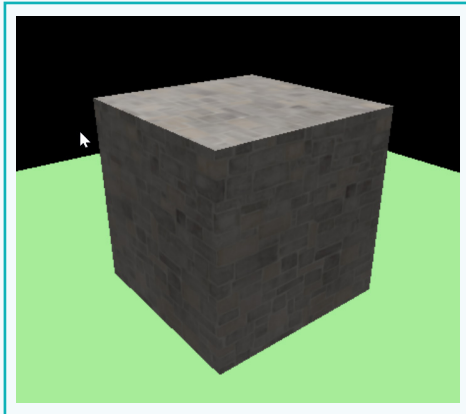
```
//Material
const boxMaterial = new THREE.MeshStandardMaterial({map:bricktexture})
boxMaterial.normalMap = bricknormaltexture; ←
const planeMaterial = new THREE.MeshBasicMaterial({color:'LightGreen'})
```

Check the output:



Activity

Compare the output with the previous step's output and observe the difference.



The texture on the cube looks realistic now, with the roughness and cracks in the bricks visible.

- The level of roughness of the material, as well as the texture, can be increased as below –

```
//Material
const boxMaterial = new THREE.MeshStandardMaterial({map:bricktexture})
boxMaterial.roughness = 2
boxMaterial.normalMap = bricknormaltexture;
boxMaterial.normalScale.set(2, 2)

const planeMaterial = new THREE.MeshBasicMaterial({color:'LightGreen'})
```

The scene's background is currently black. Change the background colour.

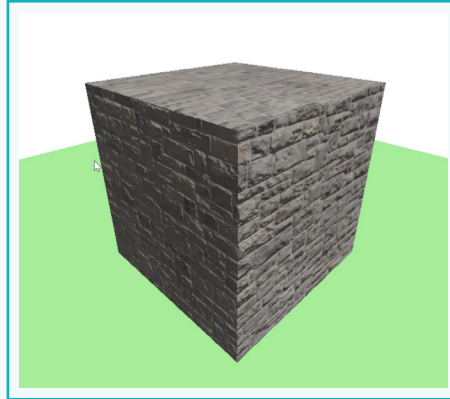
- 7 Set the background colour to any desired colour or use an image as the background.

```
//Scene
const scene = new THREE.Scene();
scene.background = new THREE.Color('White');

//Texture
const textureLoader = new THREE.TextureLoader();
```

Activity

■ Check the output:



Try:

The final touch needed for a more realistic look is Shadows. Try exploring how to add shadows to the scene, on your own.

Scan the QR code to view output:



In the next lesson, we will learn how to import 3D models and create realistic backgrounds, making our scenes come alive.

In this lesson, we revisited the basics of setting up a scene in Three.js, including adding objects, materials, and a camera. We explored how textures enhance surface details, the role of normal maps in creating depth, and how some materials require light to be rendered correctly. Additionally, we learned about scene backgrounds and their impact on the overall visual experience.

Exercise

Fill in the Blanks.

- 1 The _____ in Three.js is like the stage in a theater.
- 2 In three.js, an object is a combination of a _____ and a _____.
- 3 The _____ defines how the object looks.
- 4 The _____ calculates the position, colour, lighting, and shadows of every pixel, creating a realistic final image for display.
- 5 The normal texture is loaded using the _____ property.

Answer in One Line.

- 6 What does package.json file contain?

- 7 Which command creates the package.json file?

- 8 Which command creates the node modules folder?

- 9 Which command is used to run and test the code?

- 10 Explain what a normal map/texture is.

Exercise

 **State if the Statements are True or False.**

11 We can load textures using the Texture class.

 _____

12 The map property helps to apply a texture to a material.

 _____

13 *MeshBasicMaterial* reacts to lights in the scene.

 _____

14 Creating lights is similar to how we have created objects, materials, textures, etc.

 _____

15 It is not possible to change the colour of the background.

 _____

Tech Flash

- Scene** : It is the container that holds all the objects, lights, and cameras that make up a 3D environment.
- Geometry** : It defines the shape of objects.
- Material** : It defines the object's appearance (colour, texture, reflectivity).
- Mesh** : It is a combination of the geometry and material of an object.
- Camera** : It determines what the audience sees.
- Texture** : It is an image that covers the surface of a mesh.
- Normal Texture** : It creates the illusion of surface details like bumps, dents, and grooves without modifying the actual 3D model's geometry.